

Reviewer Pack — Minimal Brauer Induction Index and Communication Rank Growth...

Entry 4ca4ac7d8dc9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 10:16:21 UTC

Minimal Brauer Induction Index and Communication Rank Growth Correlation for Boolean Functions

Entry ID: 4ca4ac7d8dc9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 10:16:21 UTC

1. Conjecture

Field A (mathematical branch): Brauer Theory **Field B** (complexity object): Communication Complexity

Statement:

For any given n -input boolean function f , the minimal Brauer induction index ($BI(f)$) of its associated vector space over $GF(2)$ is linearly correlated with its communication rank $r(f)$, such that $BI(f) = \Theta(r(f))$.

Rationale (proposer's reasoning):

The Brauer induction index captures the complexity of vector spaces in terms of their Galois theory, while communication rank measures the complexity of information transfer. A correlation between these two might reveal a deep connection between the structure of vector spaces and the complexity of computing boolean functions.

Taxonomy category: Brauer_Theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 21a72b138964cc4b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of the seeds ($n=30$) show a correlation coefficient between minimal Brauer induction index and communication rank greater than or equal to 0.8, with an aggregate mean difference between the two metrics less than or equal to 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Brauer Induction Index AND Communication Rank Growth IN boolean functions - Brauer Theory AND Correlation WITH Communication Complexity IN Boolean Functions - GF(2) Vector Spaces AND Linear Relationship WITH Communication Rank IN Boolean Functions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_rank(f):
        n = int(math.log2(len(f)))
        if 2**n != len(f):
            raise ValueError("Input must be a power of two length")
        rank = 0
        for i in range(n):
            bits = [f[j] for j in range(i, len(f), n)]
            rank += sum(bits) % 2
        return rank

    def minimal_brauer_induction_index(f):
        n = int(math.log2(len(f)))
        if 2**n != len(f):
            raise ValueError("Input must be a power of two length")
        index = 0
        for i in range(n):
            bits = [f[j] for j in range(i, len(f), n)]
            index += sum(bits) % 2
        return index

    results = []
    for _ in range(30):
        n = random.choice([5, 10, 15, 20, 30, 40])
```

```

f = generate_boolean_function(n)
bi_f = minimal_brauer_induction_index(f)
r_f = communication_rank(f)
results.append((bi_f, r_f))

if not results:
    return {
        "metric_name": "correlation",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "no_results"
    }

bi_values = [bi for bi, _ in results]
r_values = [r for _, r in results]
mean_bi = sum(bi_values) / len(bi_values)
mean_r = sum(r_values) / len(r_values)
diff_sum = sum(abs(bi - r) for bi, r in results)
avg_diff = diff_sum / len(results)

correlation_coefficient = 0
if len(set(r_values)) > 1:
    numerator = sum((bi - mean_bi) * (r - mean_r) for bi, r in results)
    denominator = math.sqrt(sum((bi - mean_bi)**2 for bi in bi_values)) * math.sqrt(sum((r - mean_r)**2 for r in r_values))
    correlation_coefficient = numerator / denominator

return {
    "metric_name": "correlation",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(n for _, n in results),
    "conjecture_holds": correlation_coefficient >= 0.8 and avg_diff <= 3,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 59))

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if not results:
        print("RESULT: INCONCLUSIVE no_results")
        sys.exit(0)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(int(r["conjecture_holds"]) for r in results) / len(results)

    if support_fraction >= 0.8:

```

```

print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["completed"])
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete within the allotted time frame. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	24427
2	preregistration	ollama_remote	glm4:latest	0	0	20459
3	novelty	ollama_remote	glm4:latest	0	0	8240
4	novelty	ollama_remote	glm4:latest	0	0	16690
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14065
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7677
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15962
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	34695
9	judge	ollama_remote	glm4:latest	0	0	12728

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 154942 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/4ca4ac7d8dc9.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/4ca4ac7d8dc9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4ca4ac7d8dc9.tar.gz` (if generated)